# JAVA INTERVIEW PREPARATION

CHAPTER 26

DEADLOCKS, LIVENESS FAILURES, AND DIAGNOSTICS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

# Deadlocks, Liveness Failures, and Concurrency Diagnostics: When Correct Threads Stop Making Progress

Senior / Lead Java Developer Reading Chapter

Concurrency correctness is more than preventing data races. A program can preserve every value and still fail because threads cannot make progress. Deadlock, starvation, livelock, thread-pool exhaustion, permit leaks, and unbounded queues produce different symptoms and require different evidence. Senior engineers diagnose the wait graph before changing pool sizes or adding timeouts.

## Safety Versus Liveness

Safety means something bad does not happen: no lost update, corrupted state, or broken invariant. Liveness means something good eventually happens: a request completes, a lock is acquired, or a queue drains.

```
safe but not live: all account balances remain correct,
                   but every transfer thread waits forever

live but unsafe: requests complete quickly,
                 but updates are lost
```

A robust design needs both.

## The Four Deadlock Conditions

Classic deadlock requires four conditions:

- Mutual exclusion: a resource is held by one participant at a time.
- Hold and wait: a participant holds one resource while requesting another.
- No preemption: resources are released voluntarily.
- Circular wait: participants form a cycle of dependencies.

Break any one condition and that deadlock cannot occur. In application design, consistent lock ordering is often the clearest way to break circular wait.

## A Two-Lock Deadlock

```
void transfer(Account from, Account to, Money amount) {
    synchronized (from) {
        synchronized (to) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}
```

Two opposite transfers can deadlock:

```
Thread A holds Account 1 -> waits for Account 2
Thread B holds Account 2 -> waits for Account 1
                ^                 |
                +-----------------+
```

The code is locally reasonable but globally inconsistent because lock order follows argument order.

## Preventing Cycles with Lock Ordering

Define a stable total order and always acquire locks in that order.

```
void transfer(Account from, Account to, Money amount) {
    Account first = from.id().compareTo(to.id()) < 0 ? from : to;
    Account second = first == from ? to : from;

    synchronized (first) {
        synchronized (second) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}
```

The tie case must also be handled if ordering keys are not guaranteed unique. A private global tie lock can resolve identical ordering hashes, or better, order by a unique immutable ID.

Document lock levels in larger systems:

```
1. tenant lock
2. account lock
3. entry lock

never acquire a lower-numbered level while holding a higher one
```

Static policy is easier to review than attempting to infer cycles during incidents.

## Timed Acquisition Is Detection, Not Full Correctness

ReentrantLock.tryLock can avoid waiting forever and allows rollback.

```
if (!first.tryLock(100, MILLISECONDS)) return false;
try {
    if (!second.tryLock(100, MILLISECONDS)) return false;
    try {
        performTransfer();
        return true;
    } finally {
        second.unlock();
    }
} finally {
    first.unlock();
}
```

Timeout converts a permanent wait into a failure path, but partial work must be reversible and retries need backoff and idempotency. Random timeouts can turn deadlock into repeated livelock or user-visible errors. Prefer eliminating the dependency cycle when possible.

## Deadlock Beyond synchronized

Deadlocks can involve ReentrantLock, database row locks, file locks, distributed leases, futures, semaphores, and combinations across layers.

```
Thread A: holds JVM lock -> waits for database row
Thread B: holds database row -> calls code waiting for JVM lock
```

One tool may see only half the cycle. JVM thread dumps cannot show the database's complete lock graph, and database diagnostics cannot see Java monitor ownership. Correlate transaction IDs, request IDs, lock owners, and timestamps across systems.

Avoid holding an in-process lock across a remote call or database transaction when architectural semantics permit. The remote system has its own queues and locks, making the dependency graph difficult to bound.

## Thread-Pool Starvation Deadlock

No explicit monitor cycle is required. Every worker in a bounded pool can block waiting for tasks queued to the same pool.

```
Future<Result> parent = pool.submit(() -> {
    Future<Data> child = pool.submit(this::loadData);
    return transform(child.get());
});
```

```
all workers run parents
parents wait for children
children wait in pool queue
no worker is available
```

Thread dumps show workers WAITING on FutureTask rather than a monitor deadlock. The JVM deadlock detector may report no deadlock because the dependency is logical, not a recognized monitor cycle.

Prevent blocking nested dependencies in the same bounded executor. Use direct structured ownership, non-blocking composition, separate capacity only when the dependency model is sound, or virtual threads plus explicit downstream limits.

## Starvation

Starvation means a thread or task is continually denied a resource while others progress. Causes include unfair locks under heavy contention, priority misuse, a hot task monopolizing an event loop, an endless stream of higher-priority work, or pool isolation that gives one workload no capacity.

Symptoms are high tail latency for a subset rather than complete system freeze. Fair locks can help a measured lock-starvation problem but may reduce overall throughput. More often, bound task duration, partition capacity, prevent priority queues from indefinitely bypassing old work, and monitor oldest-wait age.

## Livelock

In livelock, participants actively change state but continually react to one another without completing useful work.

```
Worker A detects conflict -> releases and retries
Worker B detects conflict -> releases and retries
both retry in synchronized rhythm forever
```

Naive tryLock loops, immediate transaction retries, and collision recovery can create livelock. Add randomized exponential backoff, cap attempts, assign ownership, or impose ordering. High CPU and high retry counts with low completion distinguish livelock from threads blocked asleep.

## Permit and Resource Leaks

A missing release eventually exhausts a Semaphore or connection pool.

```
permits.acquire();
return client.call(); // exception skips release
```

Correct structure:

```
permits.acquire();
try {
    return client.call();
} finally {
    permits.release();
}
```

The same rule applies to locks, connections, streams, and temporary capacity tokens. Try-with-resources is preferable for AutoCloseable resources. Track permits in use, acquisition wait, connection borrow time, and unusually long holders.

## Blocking While Holding a Lock

Even without a cycle, long blocking inside a critical section serializes callers and looks like lock failure.

```
lock owner -> remote call waits 30 seconds
200 request threads -> BLOCKED behind owner
```

The root cause is often the remote dependency or missing timeout, while monitor contention is the amplification mechanism. Find the owner at the top of the wait chain and inspect what it is doing.

Reducing lock scope may help, but first preserve state-transition semantics. Sometimes the correct solution is a two-phase state machine, durable queue, outbox, or optimistic database transaction rather than simply moving one line outside synchronized.

## Reading a Thread Dump

A thread dump captures thread stacks and lock information at one instant.

Common states:

```
RUNNABLE       executing or runnable, sometimes inside native I/O
BLOCKED        waiting to enter a synchronized monitor
WAITING        waiting indefinitely, such as Future.get or LockSupport.park
TIMED_WAITING  waiting with a deadline, such as sleep or timed await
```

Read from the blocked thread toward the resource owner. Look for repeated identical stacks, monitor addresses, "locked" and "waiting to lock" lines, executor names, queue waits, and downstream client frames.

One dump can be misleading. Capture several dumps seconds apart. A healthy busy system changes; a stuck system often shows the same owners and waiters without progress.

The JVM can detect cycles involving owned monitors and ownable synchronizers and may print a deadlock section. Absence of that section does not exclude pool starvation, database deadlock, semaphore exhaustion, or distributed cycles.

## Java Flight Recorder and Runtime Evidence

JFR provides low-overhead events for monitor contention, thread parking, CPU samples, allocation, socket and file activity, and other runtime behavior depending on JDK configuration. It adds duration and history that a thread dump lacks.

Combine:

- Thread dumps for ownership and current stacks.
- JFR for contention duration, CPU, parking, and time sequence.
- Executor metrics for active workers, queue age, and rejection.

- Connection-pool metrics for borrow wait and active connections.
- Database lock views for row and transaction dependencies.
- Distributed traces for downstream latency and request fan-out.

Do not diagnose solely from thread state counts. Many WAITING pool workers can be normal when idle; one blocked lock owner holding a critical monitor can be catastrophic.

## High CPU Versus No CPU

Deadlock usually produces low useful CPU in the affected path because threads wait. Livelock, spin loops, failed CAS retries, and retry storms can produce high CPU.

```
low CPU + unchanged waiting stacks -> deadlock, blocked I/O, starvation
high CPU + same runnable stacks    -> loop, contention spin, livelock
high queue + all workers busy      -> saturation or downstream slowness
```

This is triage, not proof. Profile CPU, compare multiple dumps, and inspect throughput.

## Database Deadlocks

Databases can detect transaction lock cycles and abort one participant. Application code should keep transactions short, access rows in a consistent order, maintain useful indexes so updates do not lock unnecessary rows, and retry only known transient deadlock failures.

Retries require an idempotent transaction boundary, fresh transaction, bounded attempts, backoff, and monitoring. Retrying every database exception can amplify an outage or repeat non-idempotent side effects.

## Testing Liveness

Concurrency tests should use bounded completion assertions, controlled barriers, and many repetitions. Avoid proving only that one schedule worked.

```
assertTimeoutPreemptively(Duration.ofSeconds(2), () -> {
    runOpposingTransfersRepeatedly();
});
```

A timeout detects failure but may not explain it. On failure, capture thread dumps, seed values, task order, and relevant metrics. Stress tests, jcstress for memory-model questions, and production-like load tests serve different purposes.

Never rely on Thread.sleep to establish ordering. Use latches, barriers, futures, or explicit test hooks. Sleeps make tests slow and still nondeterministic.

## The Wait-For Graph Is the Unifying Mental Model

The most useful abstraction is a directed wait-for graph. A node can be a thread, task, transaction, connection, executor slot, or remote operation. An edge means that one node cannot progress until another releases a resource or produces an event.

```
request thread
    -> waits for Future
        -> child task waits in executor queue
            -> queue needs a free worker
                -> all workers wait for child Futures
```

A cycle proves deadlock only when the edges represent waits that cannot be broken by timeout, cancellation, failure, or preemption. An acyclic graph can still be unhealthy: a very long chain ending at a hung dependency

produces effective unavailability. This is why production diagnosis should first identify the wait relation and the terminal owner, then classify the liveness failure.

The graph also prevents a common mistake: focusing on the largest group of blocked threads. Two hundred request threads may be innocent waiters. The important thread is the single owner at the end of the chain, perhaps blocked in DNS, socket I/O, class initialization, or a database call while holding a monitor.

## Monitor Deadlock Versus Ownable Synchronizer Deadlock

Java distinguishes intrinsic object monitors from ownable synchronizers such as `ReentrantLock`. `ThreadMXBean.findMonitorDeadlockedThreads()` searches monitor cycles. `findDeadlockedThreads()` can include both monitors and ownable synchronizers when the JVM supports that monitoring.

```
ThreadMXBean threads = ManagementFactory.getThreadMXBean();
long[] ids = threads.findDeadlockedThreads();
if (ids != null) {
    ThreadInfo[] infos = threads.getThreadInfo(ids, true, true);
    for (ThreadInfo info : infos) {
        log.error("deadlocked thread: {}", info);
    }
}
```

These APIs are troubleshooting aids, not synchronization mechanisms. They can be relatively expensive and they cover only cycles the JVM understands. In Java 21, `ThreadMXBean` manages platform threads, not virtual threads, and its deadlock methods do not find cycles that include virtual threads. It also cannot see logical `Future` dependencies, semaphore protocols without recognized ownership, database locks, or distributed waits. A null result means "no supported cycle was found," not "the application is live."

## Reading Ownership Lines, Not Just Thread States

Consider a shortened dump:

```
"transfer-1" BLOCKED
  at Ledger.move(Ledger.java:81)
  - waiting to lock <0xB> (an Account)
  - locked <0xA> (an Account)

"transfer-2" BLOCKED
  at Ledger.move(Ledger.java:81)
  - waiting to lock <0xA> (an Account)
  - locked <0xB> (an Account)
```

The hexadecimal object identities connect waiters to owners. Start with "waiting to lock," find the thread that reports "locked" for the same identity, and repeat. For `ReentrantLock`, dumps commonly show parking through AQS plus a section of locked ownable synchronizers. For a thread waiting on a `Future`, the stack names the wait mechanism but may not identify which task must complete it; executor metrics and application correlation are then essential.

Thread state is not a diagnosis. `RUNNABLE` can include a thread in native socket work; `WAITING` can be a healthy idle worker; `BLOCKED` specifically indicates monitor entry, not every kind of lock wait. Compare stacks across time and ask whether ownership, queue position, and completed-work counters change.

## Pinning and Virtual-Thread Liveness

Virtual threads remove the need for small platform-thread pools around ordinary blocking I/O, which eliminates some pool-starvation patterns. They do not remove deadlock: a virtual thread can still participate in inconsistent lock ordering, wait on a cycle of futures, or exhaust a semaphore, connection pool, or downstream service.

When a virtual thread blocks in some situations while holding a monitor or executing native code, its carrier may remain occupied rather than being released for another virtual thread. This is called pinning. Short, infrequent pinning is not automatically a defect, but long blocking operations inside `synchronized` sections can reduce scalability. JFR pinning evidence, carrier utilization, and dependency latency help distinguish this from an ordinary logical deadlock. The architectural rule remains useful: keep critical sections small and avoid slow external operations while holding them.

## Timeout, Cancellation, and Recovery Boundaries

A timeout breaks an indefinite wait only for the caller that observes it. The underlying operation may continue holding a lock, worker, connection, or server-side transaction. If every caller times out and retries while old work continues, the system can become less live after adding timeouts.

```
deadline -> caller abandons response -> retry begins
                    |
                    +-> original database call still consumes capacity
```

Design the whole recovery boundary: propagate a remaining deadline, configure the actual I/O timeout, make cancellation cooperative, release local capacity, and decide whether the business operation may still commit. Idempotency keys or status lookup may be needed when the caller cannot know whether a timed-out remote mutation completed.

## Lock Scope Is a State-Machine Decision

"Never call I/O under a lock" is a useful warning but not a complete design. Moving I/O outside a lock can introduce a check-then-act race:

```
lock: verify order is NEW
unlock
remote charge succeeds
lock: another thread already cancelled order
```

The robust alternative may be an explicit state transition such as `NEW -> PAYMENT_PENDING`, followed by I/O without the lock, and then a conditional transition to `PAID` or `PAYMENT_FAILED`. Distributed workflows may require an outbox, idempotent consumer, compensation, or reconciliation. Reducing lock duration is safe only when the new state machine preserves the invariant under every interleaving and failure.

## A Strong Interview Explanation

Start by separating safety from liveness. Deadlock is a cycle in a wait-for graph under mutual exclusion, hold-and-wait, and non-preemptible ownership; global lock ordering removes circular wait and is generally stronger than relying on timeouts. Then distinguish monitor deadlock, ownable-synchronizer deadlock, pool starvation, starvation, livelock, and resource exhaustion because their evidence differs.

For diagnosis, correlate repeated thread dumps with JFR, executor queue age, connection-pool waits, database lock graphs, and traces. Follow each waiter to the owner or terminal dependency. State the limits of JVM detection, including logical and cross-system waits and Java 21 virtual-thread coverage. Finally explain recovery semantics: deadlines and cancellation must reach the real operation, retries must be bounded and idempotent, and a restart is mitigation rather than a correction of the dependency design.

## A Production Investigation Sequence

When requests stop completing:

1. Establish scope, start time, and affected endpoints. 2. Compare throughput, latency, CPU, queue age, and dependency metrics. 3. Capture multiple thread dumps and a short JFR recording when safe. 4. Identify waiters, owners, and the oldest resource hold. 5. Check database locks, connection pools, executors, and remote timeouts. 6. Mitigate safely: shed load, stop retries, isolate traffic, or restart only with awareness that evidence and in-flight work may be lost. 7. Fix the dependency design and add detection, not merely a larger pool.

Increasing threads can worsen a downstream bottleneck and increase retained requests. A restart clears symptoms but does not remove the cycle or leak.

## Interview-Ready Summary

Deadlock is a circular wait under mutual exclusion, hold-and-wait, and no preemption. Consistent global lock ordering is a strong prevention technique. Timed acquisition provides a failure path but requires rollback and does not replace sound ordering.

Liveness failures also include pool starvation, starvation, livelock, resource leaks, and long blocking while holding a lock. JVM deadlock detection covers only some cycles. Diagnose with repeated thread dumps, JFR, executor and pool metrics, database lock graphs, and traces. Follow the dependency chain to the owner rather than increasing capacity blindly.

## Revision Notes

- Safety protects invariants; liveness guarantees eventual progress.
- Deadlock requires mutual exclusion, hold-and-wait, no preemption, and circular wait.
- Break one required condition to prevent a particular deadlock.
- Acquire multiple locks in a stable global order.
- Lock ordering keys must be immutable and uniquely tie-broken.
- tryLock timeouts create failure paths but require rollback and policy.
- Deadlocks can cross JVM, database, file, and distributed resources.
- Avoid holding local locks across remote or database waits when possible.
- Bounded pools can starve when parents wait for queued child tasks.
- Logical pool deadlocks may not appear in JVM deadlock detection.
- Starvation affects some work while other work progresses.
- Livelock consumes activity without useful completion.
- Randomized backoff and ownership can break retry synchronization.
- Release locks, permits, and connections in finally or try-with-resources.
- Long I/O under a lock amplifies dependency slowness.
- Capture multiple thread dumps, not just one.
- BLOCKED means monitor entry; WAITING can have many causes.
- JFR adds duration and history to stack evidence.
- Database deadlock retries must be bounded, fresh, and idempotent.
- Do not use Thread.sleep as a concurrency ordering mechanism in tests.
- A restart mitigates symptoms but does not fix the liveness design.
- Model liveness as a wait-for graph and follow edges to the terminal owner.
- Many waiters can be symptoms of one slow lock or dependency owner.
- findMonitorDeadlockedThreads covers monitors; findDeadlockedThreads can add ownable synchronizers.
- Java 21 ThreadMXBean does not monitor virtual threads.
- A null deadlock result does not exclude logical or cross-system deadlock.
- Thread state labels require stack, ownership, and time-series context.
- Virtual threads reduce some pool starvation but do not remove resource cycles.
- Timeout of a caller does not guarantee termination of underlying work.
- Moving I/O outside a lock requires an explicit safe state transition.